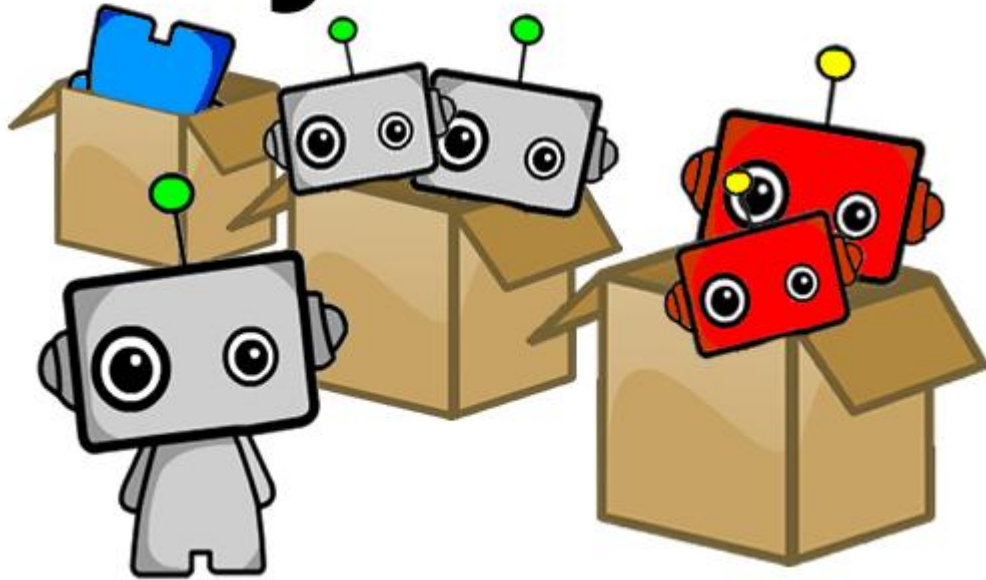


Sorting Algorithms



Sortieren und Suchen

Programmiertechnik B

Elektrotechniker/-in HF

Lukas Müller

1 Einleitung und Begriffe



Lernziele

Nach der Bearbeitung dieses Kapitels

- können Sie die zentralen Begriffe des Themenfeldes Sortierung erklären.

Zu Beginn eine kleine Geschichte:



Der Mistkäfer Willi möchte Ordnung in seine Mistkugelsammlung bringen. Am liebsten hätte er sie schön der Grösse nach sortiert, die kleinste Kugel links, die grösste rechts. Leider hat Willi keine Ahnung, wie er das anstellen könnte. Er setzt sich auf eine der Mistkugeln und überlegt, aber es fällt ihm keine Lösung ein. Da wählt er in seinem Frust zufällig zwei Mistkugeln aus, bei denen die Reihenfolge nicht stimmt, und vertauscht die beiden. Willi ist natürlich klar, dass er damit seine Kugelreihe noch lange

nicht sortiert hat. In seiner wachsenden Verzweiflung wählt er ein weiteres Kugelpaar, das nicht richtig geordnet ist, und vertauscht auch dieses. Diesen Vorgang wiederholt er nun laufend. Nach vielen solchen Vertauschungen stutzt Willi plötzlich: Er findet kein einziges "falsches" Paar mehr! Enttäuscht wendet er sich ab und versinkt wieder ins Grübeln. Nach einer Weile schaut er auf und betrachtet die Reihe seiner Mistkugeln...

1.1 Wozu sortieren?

Wir sortieren Farbstifte, Rechnungen, Adressen, Kleider, Bankbelege, Personen und vieles mehr. Wozu eigentlich? Die Antwort ist klar: Sortieren erleichtert das Wiederfinden. Stellen wir uns vor, im Telefonbuch wären die Einträge in zufälliger Reihenfolge abgedruckt. Das Telefonbuch wäre nahezu nutzlos.

Der Nutzen der Sortierung ist also wohl unbestritten. Für Informatiker interessant ist die Tatsache, dass es sehr viele verschiedene Sortierverfahren gibt. Diese sind keineswegs gleichwertig, sondern weisen gewaltige Unterschiede bezüglich ihrer Effizienz auf.

Wir werden in diesem Kapitel drei Sortierverfahren kennen lernen und implementieren. Ausserdem werden wir ein Mass für die Effizienz dieser Verfahren einführen und so in der Lage sein, sie miteinander zu vergleichen.



1.1.1 Aufgabe

Beschreiben Sie den Unterschied zwischen Inhaltsverzeichnis und Index eines Sachbuches bezüglich der Reihenfolge der Einträge.

1.2 Sortierkriterium und Vergleichsfunktion

Je nach Anwendungsgebiet sind verschiedene Sortierkriterien denkbar: Rechnungen sortieren wir nach Datum oder nach Absender, Bankbelege nach Datum oder nach Bank. Eine Gruppe von Personen kann man z.B. nach Grösse, Gewicht oder nach Alter sortieren. Farbstifte können wir z.B. nach Helligkeit sortieren, nach Rotanteil oder gemäss der Reihenfolge, die auf der Farbstiftschachtel aufgedruckt ist.

Es gibt auch kombinierte Sortierkriterien. Nehmen wir zum Beispiel ein Telefonbuch. Die Einträge sind in erster Linie nach der Ortschaft sortiert, innerhalb einer Ortschaft nach Name sortiert, bei gleichen Namen nach Vorname.

Die Wahl des Kriteriums ist entscheidend für die Nützlichkeit der Sortierung. Oder wer könnte wohl etwas anfangen mit einem Telefonbuch, das nach Telefonnummern sortiert ist?

Um eine Liste nach einem bestimmten Kriterium sortieren zu können, müssen je zwei Elemente gemäss diesem Kriterium mit einander verglichen werden können. Von zwei beliebigen Elementen muss entscheidbar sein, welches das grössere bzw. das kleinere ist. Natürlich gibt es auch den Fall der Gleichheit zweier Elemente.

Diese Vergleichbarkeit ist z.B. bei Zahlen selbstverständlich. Bei Buchstaben ebenfalls, da gilt die alphabetische Ordnung. Der Vergleich zweier Buchstabenfolgen (also Wörter) gemäss alphabetischer Ordnung ist hingegen schon etwas komplizierter.

Wir werden uns hier auf das Sortieren von Arrays ganzer Zahlen beschränken.



1.2.1 Aufgabe

In welcher Reihenfolge stehen folgende Namen im Telefonbuch?

- Vontobel
- Vogel
- Völki
- Vock
- Vollenweider
- von Salis
- Vögeli



1.2.2 Aufgabe

Welche Probleme ergeben sich, wenn man eine Schulklasse nach

- a) Haarfarbe
- b) Geschlecht
- c) Hobbies

sortieren möchte?

1.3 Sortierrichtung

Sortieren ist immer in zwei Richtungen möglich: Bei aufsteigender Sortierung kommt das kleinste Element zuerst, das grösste zuletzt. Bei absteigender Sortierung ist es genau umgekehrt.

Wir werden hier praktisch ausschliesslich aufsteigend sortieren.

1.4 Wann ist ein Array sortiert?

Bei dieser Frage hilft uns die Geschichte des Mistkäfers Willi. Nachdem er viele Paare vertauscht hat, bei denen die linke Kugel grösser war als die rechte, fand er irgendwann kein solches "falsches" Paar mehr. Natürlich war zu diesem Zeitpunkt die Reihe vollständig sortiert.

Warum funktioniert dieses Verfahren eigentlich? Nun, jede Vertauschung bringt eine grössere Kugel ein Stück nach rechts und eine kleinere Kugel ein Stück nach links. Auf diese Weise trägt jede Vertauschung ein kleines Stück zur Sortierung bei. Jede Vertauschung macht also die Sortierung ein bisschen besser. Die Sortierung ist perfekt, wenn es keine falschen Paare mehr gibt.

Wir halten also fest: Ein Array ist sortiert, wenn es keine zwei Elemente mit falscher Reihenfolge gibt.

Es ist leicht einzusehen, dass auch die folgende Formulierung gilt: Ein Array ist sortiert, wenn es keine zwei *benachbarten* Elemente mit falscher Reihenfolge gibt.



1.4.1 Aufgabe

Programmieren Sie eine Funktion

```
bool istSortiert(int[] a)
```

welche herausfindet, ob ein gegebener Array sortiert ist. Testen Sie die Funktion mit verschiedenen Arrays.

1.5 Begriff des Algorithmus

Unter einem **Algorithmus** versteht man ein Problemlösungsverfahren mit folgenden Merkmalen (sinngemäss aus *J. Hromkovi : Sieben Wunder der Informatik*):

- Das Verfahren muss so einfach und präzise formuliert sein, dass es auch von jemandem angewendet werden kann, der kein Experte für das zu lösende Problem ist.
- Das Verfahren muss nicht nur einen einzelnen Problemfall lösen können, sondern alle Fälle eines bestimmten Problemtyps.
- Das Verfahren muss in jedem Fall in endlicher Zeit das korrekte Ergebnis liefern.

Beispiele von Algorithmen aus dem Alltag sind Kochrezepte oder eine Bauanleitung für ein Modellflugzeug.

Algorithmen kann man in natürlicher Sprache oder in speziell dafür entwickelten Ablaufdiagrammen formulieren. Für Informatiker sind natürlich vor allem solche Algorithmen von Interesse, welche sich mit Hilfe einer Programmiersprache auf einem Computer umsetzen lassen.



1.5.1 Aufgabe

Nennen Sie zwei weitere Beispiele von Algorithmen aus dem Alltag.

1.6 Zusammenfassung

In diesem Abschnitt haben uns zunächst in Erinnerung gerufen, welche wichtige Bedeutung sortierte Daten in unserem Alltag haben.

Wenn wir Daten oder Gegenstände sortieren wollen, müssen wir uns stets überlegen, nach welchem *Kriterium* dies geschehen soll. Ein Merkmal ist nur dann als Sortierkriterium geeignet, wenn zwei beliebige Elemente bezüglich dieses Merkmals miteinander *vergleichbar* sind.

In den folgenden Abschnitten werden wir verschiedene Sortierverfahren entwickeln. Um diese Verfahren auf dem Computer implementieren zu können, müssen wir sie *präzise* formulieren. Wir werden auch darauf achten, dass sie möglichst *allgemeingültig* sind. Das bedeutet, dass ein Verfahren Zahlenreihen beliebiger Länge sortieren können muss, und zwar unabhängig davon, in welcher Reihenfolge die Zahlen vor der Sortierung angeordnet sind. Ausserdem müssen unsere Verfahren in jedem Fall in *endlicher Zeit* die Lösung finden. Diese gewünschten Merkmale unserer Sortierverfahren haben wir unter dem Begriff *Algorithmus* zusammengefasst.

2 BubbleSort



Lernziele

Nach der Bearbeitung dieses Kapitels

- verstehen Sie die Funktionsweise eines einfachen Sortierverfahrens namens **BubbleSort**.
- können Sie dieses Sortierverfahren und Varianten davon programmieren.
- kennen Sie eine Methode, wie man die "Geschwindigkeit" eines Sortierverfahrens messen kann, und können entsprechenden Code in ihr Programm einbauen.

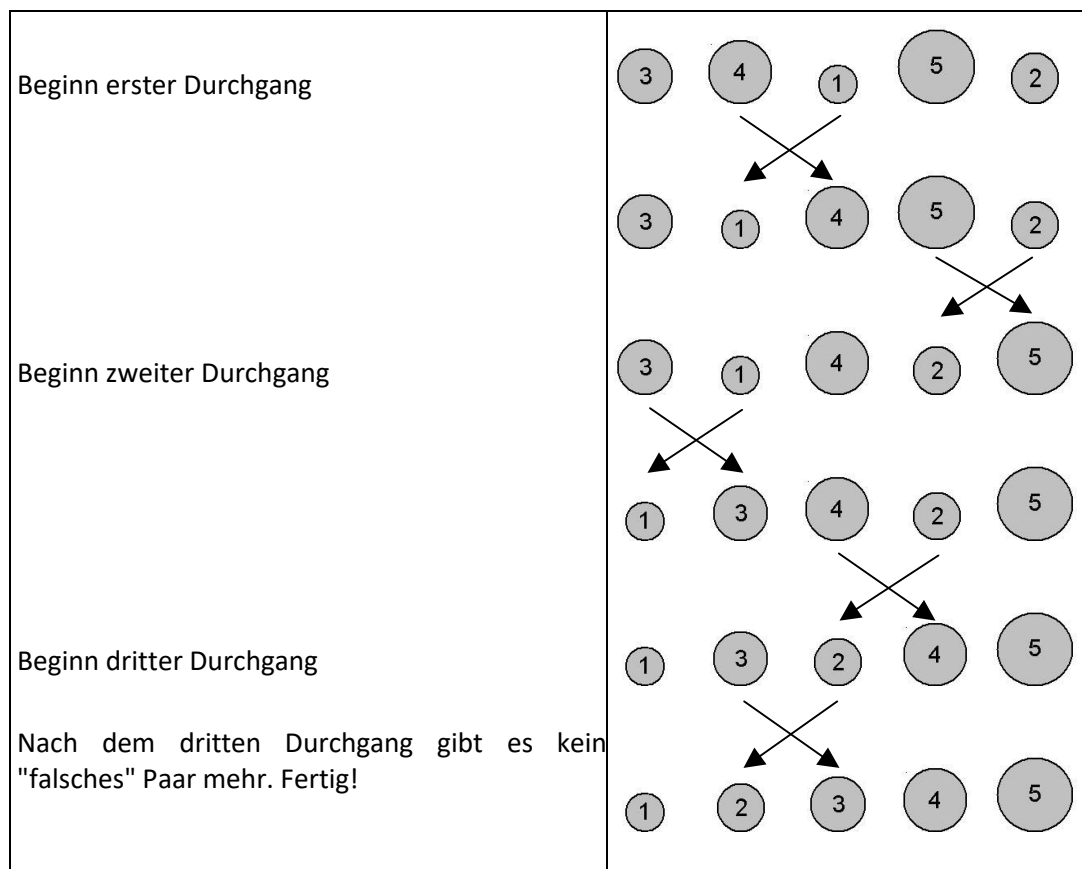
2.1 Die Idee



Oje, Willi hat seine Mistkugeln auf offenem Feld aufgereiht. Ein heftiger Windstoss und die Kugeln rollen davon und liegen schon wieder in einem Durcheinander. Also nochmals sortieren. Willi ist noch ganz erschöpft von seiner ersten Sortier-Aktion. Vor allem das Suchen der "falschen" Paare war sehr anstrengend! Er musste dabei immer die ganze Kugelreihe überblicken. Gut, ich machs nochmals, sagt er sich, aber diesmal ein bisschen systematischer. Er sucht die "falschen" Paare jetzt nicht mehr

nach dem Zufallsprinzip, sondern geht von links nach rechts durch die Reihe. Dabei vergleicht er immer zwei benachbarte Kugeln. Sobald er zwei Kugeln mit falscher Reihenfolge gefunden hat, vertauscht er diese. Nachdem er die ganze Reihe so abgearbeitet hat, beginnt er wieder von vorne, d.h. von links. Wenn es schon mit Zufall geklappt hat, denkt sich Willi, dann muss es doch mit System erst recht klappen. Vielleicht sogar noch schneller als vorher? Tatsächlich findet er nach einigen Durchgängen kein "falsches" Paar mehr...

Wir spielen ein Beispiel durch. Der Einfachheit halber nehmen wir an, dass alle Kugeln verschieden gross sind und entsprechend ihrer Grösse beschriftet sind:



Schauen wir uns an, was genau passiert ist:

Erster Durchgang: Zuerst wurde die 4 mit der 1 vertauscht, dann noch die 5 mit der zwei. Am Ende des ersten Durchgangs liegt also die 5 ganz rechts

Zweiter Durchgang: Zuerst wird die 3 mit der 1 vertauscht, dann noch die 4 mit der 2. Am Ende des zweiten Durchgangs liegt also die 4 an der zweiten Stelle von rechts.

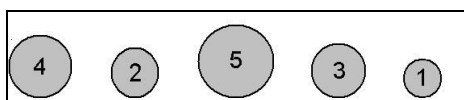
Dritter Durchgang: Die 3 wird mit der 2 vertauscht. Am Ende des dritten Durchgangs liegt also die 3 an der dritten Stelle von rechts.

Wenn man sich anstelle der Mistkugeln Luftblasen in einem Aquarium vorstellt und das Ganze um 90° gegen den Uhrzeigersinn dreht, sieht man in Gedanken, wie die *Luftblasen* im Wasser *aufsteigen*. Zuerst die grösste ganz nach oben, dann die zweitgrösste usw. Die Vorstellung von den steigenden Luftblasen hat diesem Sortierverfahren seinen Namen gegeben: **BubbleSort** (*bubble* (engl.) = Blase).



2.1.1 Aufgabe (2.1)

Sortieren Sie folgende Reihe mit dem BubbleSort-Verfahren. Füllen Sie dazu wie im obigen Beispiel eine Tabelle aus.



2.2 Der Algorithmus

Wir sehen im Beispiel auf Seite 7 sofort, dass die Reihe nach dem dritten Durchgang sortiert ist und wir damit fertig sind. Wie aber kann ein Programm herausfinden, ob eine Reihe sortiert ist? Richtig: Es muss alle Elemente durchgehen und "falsche" Paare finden (siehe Einleitung, Abschnitt "Wann ist ein Array sortiert?"). Das ist aber genau das, was in einem Durchgang geschieht. Um zu entscheiden, ob die Reihe sortiert ist, kann unser Programm also einfach einen weiteren Durchgang ausführen. Wenn in einem Durchgang kein "falsches" Paar mehr gefunden wird, sind wir fertig.

Diese Überlegungen führen uns auf folgenden Algorithmus:



Algorithmus

Schritt	Was tun
(1)	Wähle die ersten beiden Elemente von links.
(2)	Ist beim gewählten Paar das linke Element grösser als das rechte? Wenn ja, vertausche die beiden.
(3)	Hat es rechts des soeben untersuchten Paares noch weitere Kugeln? Wenn ja, wähle das nächste Paar benachbarter Elemente und gehe zu (2).
(4)	Wir haben einen Durchgang beendet. Wurde in diesem Durchgang mindestens ein "falsches" Paar gefunden und vertauscht? Wenn ja, gehe zu (1), d.h. starte einen weiteren Durchgang. Wenn nein, sind wir fertig.

Das Beispiel von Seite 7 mit Zahlen und streng nach Anleitung:

Schritt	Was tun	Ergebnis dieses Schrittes				
	Ausgangslage	3	4	1	5	2
(1)	Wähle das erste Paar: 3 und 4	3	4	1	5	2
(2)	3 ist nicht grösser als 4, also nichts tun	3	4	1	5	2
(3)	Wähle nächstes Paar: 4 und 1	3	4	1	5	2
(2)	4 ist grösser als 1, also vertauschen	3	1	4	5	2
(3)	Wähle nächstes Paar: 4 und 5	3	1	4	5	2
(2)	4 ist nicht grösser als 5, also nichts tun	3	1	4	5	2
(3)	Wähle nächstes Paar: 5 und 2	3	1	4	5	2
(2)	5 ist grösser als 2, also vertauschen	3	1	4	2	5
(3)	Es hat weiter rechts kein Paar mehr.	3	1	4	2	5
(4)	In diesem Durchgang wurden 2 Vertauschungen gemacht, also noch ein Durchgang.	3	1	4	2	5
(1)	Wähle das erste Paar: 3 und 1	3	1	4	2	5
(2)	3 ist grösser als 1, also vertauschen	1	3	4	2	5
(3)	Wähle nächstes Paar: 3 und 4	1	3	4	2	6
(2)	3 ist nicht grösser als 4, also nichts tun	1	3	4	2	5
(3)	Wähle nächstes Paar: 4 und 2	1	3	4	2	5
(2)	4 ist grösser als 2, also vertauschen	1	3	2	4	5
(3)	Wähle nächstes Paar: 4 und 5	1	3	2	4	5
(2)	4 ist nicht grösser als 5, also nichts tun	1	3	2	4	5
(3)	Es hat weiter rechts kein Paar mehr.	1	3	2	4	5
(4)	In diesem Durchgang wurden 2 Vertauschungen gemacht, also noch ein Durchgang.	1	3	2	4	5
(1)	Wähle das erste Paar: 1 und 3	1	3	2	4	5
(2)	1 ist nicht grösser als 3, also nichts tun	1	3	2	4	5
(3)	Wähle nächstes Paar: 3 und 2	1	3	2	4	5
(2)	3 ist grösser als 2, also vertauschen	1	2	3	4	5
(3)	Wähle nächstes Paar: 3 und 4	1	2	3	4	5
(2)	3 ist nicht grösser als 4, also nichts tun	1	2	3	4	5
(3)	Wähle nächstes Paar: 4 und 5	1	2	3	4	5
(2)	4 ist nicht grösser als 5, also nichts tun	1	2	3	4	5
(3)	Es hat weiter rechts kein Paar mehr.	1	2	3	4	5
(4)	In diesem Durchgang wurde 1 Vertauschung gemacht, also noch ein Durchgang.	1	2	3	4	5
(1)	Wähle das erste Paar: 1 und 2	1	2	3	4	5
(2)	1 ist nicht grösser als 2, also nichts tun	1	2	3	4	5
(3)	Wähle nächstes Paar: 2 und 3	1	2	3	4	5
(2)	2 ist nicht grösser als 3, also nichts tun	1	2	3	4	5
(3)	Wähle nächstes Paar: 3 und 4	1	2	3	4	5
(2)	3 ist nicht grösser als 4, also nichts tun	1	2	3	4	5
(3)	Wähle nächstes Paar: 4 und 5	1	2	3	4	5
(2)	4 ist nicht grösser als 5, also nichts tun	1	2	3	4	5
(3)	Es hat weiter rechts kein Paar mehr.	1	2	3	4	5
(4)	In diesem Durchgang wurden keine Vertauschungen gemacht, also sind wir fertig.	1	2	3	4	5
	Fertig	1	2	3	4	5



2.2.1 Aufgabe

Spiele Sie den BubbleSort-Algorithmus an folgendem Zahlenbeispiel durch. Halten Sie sich dabei genau an die Anleitung und füllen sie wie im obigen Beispiel eine Tabelle aus.

11 3 6 7 3 6



2.2.2 Aufgabe

An dieser Stelle wollen wir das Gerüst für ein grösseres Programm erstellen. Dieses Programm werden wir nach und nach erweitern, indem wir alle hier behandelten Sortierverfahren einbauen.

Erzeugen Sie zunächst in einem neuen Windows-Projekt folgendes Formular:

Klick auf einen "Hinzufügen"-Button soll einen bzw. mehrere Einträge zu der linken ListBox hinzufügen. Klick auf den "Alle löschen"-Button soll die linke ListBox leeren. Programmieren Sie die Buttons entsprechend.



2.2.3 Aufgabe

Erstellen Sie in ihrem Programm eine Funktion

```
void bubbleSort(int[] a)
```

welche den BubbleSort-Algorithmus gemäss obiger Anleitung implementiert. Verwenden Sie Schleifen, keine goto-Befehle.

Erzeugen Sie im Formular zwischen den beiden Listboxen einen Button "BubbleSort", welcher aus dem Inhalt der linken ListBox einen Array von Zahlen erzeugt, damit die Funktion bubbleSort() aufruft und den sortierten Array in der rechten Listbox ausgibt.

Tipp: Erstellen Sie Funktionen für die Erzeugung eines Arrays aus der linken ListBox sowie für die Ausgabe eines Arrays in die rechte ListBox. Wir werden sie noch öfters brauchen.

Testen Sie ihr erstes Sortierprogramm mit verschiedenen Arrays!



2.2.4 Aufgabe

Was passiert, wenn man im Schritt 2 des BubbleSort-Algorithmus den Begriff "grösser als" durch a) "kleiner als"

- b) "grösser als oder gleich"
- c) "kleiner als oder gleich" ersetzt?

2.3 Zusammenfassung

Mit **BubbleSort** haben wir ein einfaches Sortierverfahren kennengelernt und implementiert. Gleichzeitig haben wir auch Vorbereitungen für die folgenden Kapitel getroffen, indem wir ein Programmgerüst entwickelt haben, in welches wir auch weitere Sortiermethoden einbauen können.

Als Mass für den Aufwand eines Sortier-Algorithmus haben wir die Anzahl Vergleiche und Vertauschungen eingeführt und in unser Programm eingebaut. Mit diesem Instrument werden wir der Lage sein, den Aufwand verschiedener Verfahren miteinander zu vergleichen. Wir haben bereits erfahren, wie sich einfache Optimierungen von **BubbleSort** günstig auf die Laufzeit auswirken.

3 SelectionSort



Lernziele

Nach der Bearbeitung dieses Kapitels

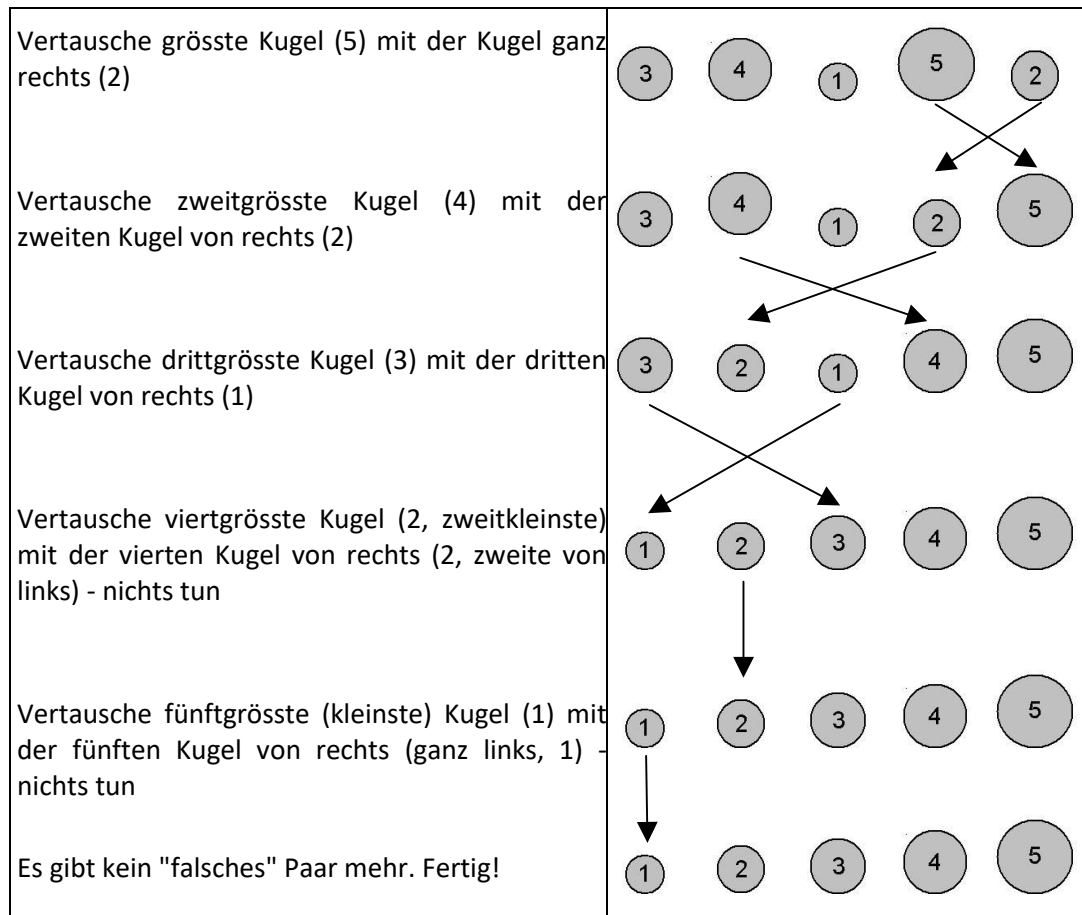
- verstehen Sie die Funktionsweise eines Sortierverfahrens namens SelectionSort.
- können Sie dieses Sortierverfahren und Varianten davon programmieren.

3.1 Die Idee



Ein furchtbares Gewitter ist durch das Land gezogen und hat Willis fein säuberlich sortierte Mistkugelsammlung total durcheinandergebracht. Diesmal ist Willi aber guten Mutes. Er weiss ja nun, wie er die Kugeln wieder in die richtige Reihenfolge bringen kann. Schon will er ans Werk gehen. Aber halt! Die Kugeln sind vom Regen noch ganz schmutzig. Muss er wirklich jede Kugel so oft anfassen wie beim ersten Mal? Nicht dass er sich vor seinen Mistkugeln ekeln würde, aber die Kugeln könnten dabei Schaden nehmen. Er setzt sich hin und überlegt. Am Schluss muss doch die grösste Kugel ganz rechts liegen, das ist klar. Warum suche ich nicht einfach die grösste Kugel und lege sie ganz nach rechts? Gedacht, getan. Das ist aber gar nicht so einfach, denn die beiden grössten Kugeln sind fast gleich gross. Willi nimmt sein Messband und misst die Umfänge der beiden Kandidaten. Das Resultat ist eindeutig. Er bringt also die grösste Kugel an die Position ganz rechts, indem er sie mit der Kugel vertauscht, welche vorher dort war. Die zweitgrösste Kugel bekommt den Platz direkt links neben der grössten, ebenfalls durch Vertauschen. So fährt Willi nun fort: Er bestimmt die drittgrösste Kugel und bringt sie an die dritte Position von rechts und so weiter. Es dauert gar nicht lange, bis er fertig ist. Zufrieden schaut er sein Werk an und überlegt sich, wievielmals er jede Kugel anfassen musste...

Unser Beispiel aus dem ersten Kapitel, sortiert mit Willis neuen Verfahren:



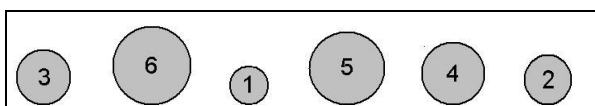
In diesem Beispiel sind wir mit drei Vertauschungen ausgekommen. Die letzten beiden Kugeln waren schon am richtigen Ort. War das Zufall? Bei der vorletzten schon, aber bei der letzten nicht. Wenn nämlich alle Kugeln ausser die kleinste an ihren richtigen Platz verschoben wurden, dann muss die kleinste zwangsläufig auch schon an ihrem richtigen Platz stehen, nämlich ganz links.

Dieses Verfahren trägt den Namen **SelectionSort**, weil in jedem Durchgang das nächstgrösste Element des Arrays *ausgewählt* und an seinen endgültigen Platz gesetzt wird (*selection (engl.) = Auswahl*).



3.1.1 Aufgabe

Sortieren Sie folgende Reihe mit dem SelectionSort-Verfahren. Füllen Sie dazu wie im obigen Beispiel eine Tabelle aus. Lassen Sie den letzten, unnötigen Durchgang weg.



3.3 Der Algorithmus

Den Algorithmus für einen Array mit n Elementen könnte man so formulieren:



Algorithmus

Schritt	Was tun
(1)	Suche das grösste Element des ganzen Arrays und vertausche es mit dem Element ganz rechts.
(2)	Suche das zweitgrösste Element des ganzen Arrays und vertausche es mit dem zweiten Element von rechts.
...	... und so weiter...
($n-1$)	Suche das zweitkleinste Element und vertausche es mit dem zweiten Element von links.

Wir wollen uns aber mit dieser saloppen "und so weiter"-Formulierung nicht zufrieden geben. Präziser ist die folgende Darstellung:



Algorithmus

Schritt	Was tun
(1)	Setze einen Zähler i auf 1.
(2)	Suche das i -t-grösste Element (*).
(3)	Vertausche es mit dem i -ten Element von rechts.
(4)	Erhöhe die den Zähler i um 1. Wenn i kleiner als die Arraylänge ist, gehe zu (2), sonst sind wir fertig.

(*) mit dem " i -grössten Element" ist folgendes gemeint:

i	" i -t-grösstes Element"
1	grösstes
2	zweitgrösstes
3	drittgrösstes
...	...

Willis Beispiel:

Schritt	Was tun	i	Ergebnis dieses Schrittes				
	Ausgangslage		3	4	1	5	2
(1)	Setze i auf 1	1	3	4	1	5	2
(2)	Suche das grösste Element: 5	1	3	4	1	5	2
(3)	Vertausche es mit dem Element ganz rechts: 2	1	3	4	1	2	5
(4)	Erhöhe i um 1	2	3	4	1	2	5
(2)	Suche das zweitgrösste Element: 4	2	3	4	1	2	5
(3)	Vertausche es mit dem zweiten Element von rechts: 2	2	3	2	1	4	5
(4)	Erhöhe i um 1	3	3	2	1	4	5
(2)	Suche das drittgrösste Element: 3	3	3	2	1	4	5
(3)	Vertausche es mit dem dritten Element von rechts: 1	3	1	2	3	4	5
(4)	Erhöhe i um 1	4	1	2	3	4	5
(2)	Suche das viertgrösste Element: 2	4	1	2	3	4	5
(3)	Vertausche es mit dem vierten Element von rechts (Vertauschung mit sich selber, d.h. nichts tun)	4	1	2	3	4	5
(4)	Erhöhe i um 1. i ist nicht kleiner als die Arraylänge, also sind wir fertig.	5	1	2	3	4	5
	Fertig		1	2	3	4	5

Bevor wir diesen Algorithmus programmieren, wollen wir noch ein wichtiges Detail klären: Im Schritt (2) müssen wir das i-t-grösste Element des Arrays bestimmen. Dieses Problem allgemein zu lösen ist gar nicht so einfach. Glücklicherweise ist es aber in unserem Fall nicht schwierig.

Schauen wir uns nochmals den obigen Ablauf an: Zu Beginn des zweiten Durchgangs ist das grösste Element ganz rechts, zu Beginn des dritten Durchgangs ist zusätzlich das zweitgrösste an zweiter Stelle von rechts. Allgemein: Zu Beginn des Durchgangs i befinden sich die (i-1) grössten Elemente auf der rechten Seite des Arrays, und zwar bereits in sortierter Reihenfolge und damit an ihrer endgültigen Position. An diesem Teil des Arrays müssen wir also gar nichts mehr ändern. Dieser bereits sortierte Bereich ist in der obigen Tabelle grau schraffiert. Wie finden wir nun das i-t-grösste Element? Ganz einfach: Es ist das grösste Element des linken, noch unsortierten Teils unseres Arrays.

Hier die vorläufig letzte Version unseres Algorithmus. Sie sieht vielleicht ein bisschen komplizierter aus als die vorherige, ist aber nachher einfacher zu programmieren:



Algorithmus

Schritt	Was tun
(1)	Setze den Zähler i auf 1.
(2)	Suche das grösste Element der (n-i+1) Elemente von links.
(3)	Vertausche es mit dem i-ten Element von rechts.
(4)	Erhöhe den Zähler i um 1. Wenn i kleiner als die Arraylänge ist, gehe zu (2), sonst sind wir fertig.



3.3.1 Aufgabe

Spielen Sie den SelectionSort-Algorithmus an folgendem Zahlenbeispiel durch. Halten Sie sich dabei genau an die Anleitung und füllen sie wie im obigen Beispiel eine Tabelle aus.

9 3 8 8 2 6



3.3.2 Aufgabe

Implementieren Sie den SelectionSort-Algorithmus gemäss obiger Ableitung und bauen Sie ihn in ihr Programm ein. Verwenden Sie Schleifen und keine goto Anweisungen. Bauen Sie von Anfang an die Zählung der Anzahl Vergleiche und Vertauschungen ein.



3.3.3 Zusatzaufgabe

- Wie oft wird das grösste Element bei SelectionSort höchstens vertauscht? Wie oft das kleinste?
- Wievielmals wird ein Element im Durchschnitt vertauscht?

Hinweis: Gehen Sie bei dieser Aufgabe davon aus, dass Vertauschungen "stur" ausgeführt werden, d.h. es gibt auch unnötige Vertauschungen eines Elements mit sich selber. Diese werden ebenfalls gezählt.

3.4 Aufwandmessungen



Aufgabe

Wenden Sie SelectionSort auf Zufalls-Arrays verschiedener Grössen an und tragen Sie die Anzahl Vergleiche und Vertauschungen in eine Tabelle ein.

Anzahl Vergleiche					
Sortierverfahren	Arraygrösse				
	10	30	100	300	1000
SelectionSort					

Anzahl Vertauschungen					
Sortierverfahren	Arraygrösse				
	10	30	100	300	1000
SelectionSort					



3.4.1 Zusatzaufgabe

Folgender naheliegender Gedanke führt uns zu einer rekursiven Variante von SelectionSort: Man suche das grösste Element des Arrays und bringe es an die Position ganz rechts. Anschliessend sortiere man den verbleibenden Array, also ohne das grösste Element.

Implementieren Sie die rekursive Variante von SelectionSort. Für eine einfache Anwendbarkeit empfiehlt es sich, zwei Funktionen zu erstellen:

Die rekursive Funktion

```
void selectionSortRek(int[] a, int n)
```

implementiert den eigentlichen rekursiven SelectionSort-Algorithmus. Diese Funktion hat zwei Parameter: den Array a und die Länge n, bis zu der sortiert werden soll.

Die Funktion

```
void selectionSort3(int[] a)
```

kann von aussen ganz normal aufgerufen werden. Sie enthält nur eine einzige Anweisung, nämlich den Aufruf von selectionSortRek() mit dem ganzen Array a.



3.4.2 Zusatzaufgabe

In der Praxis müssen häufig Listen sortiert werden, welche nicht Zahlen enthalten, sondern andere, unter Umständen zusammengesetzte Elemente.

Erstellen Sie ein Programm, mit dem der Benutzer eine Liste von Adressen erfassen und diese nach Name, Postleitzahl oder Ort sortieren kann.

3.5 Zusammenfassung

SelectionSort stellt im Vergleich zu BubbleSort eine wesentliche Verbesserung in Bezug auf die Laufzeit dar. Der Grund liegt darin, dass bei SelectionSort ein Element direkt an seinen definitiven Platz gesetzt wird, anstatt wie bei BubbleSort durch fortlaufendes Vertauschen benachbarter Elemente schrittweise dorthin gebracht zu werden. Wir haben gesehen, wie dadurch die Anzahl Vertauschungen dramatisch reduziert wird.

Trotz der massiven Verbesserung ist SelectionSort gegenüber BubbleSort nicht etwa komplizierter oder schwerer zu verstehen. Im Gegenteil: Für viele dürfte die Funktionsweise von SelectionSort sogar noch einleuchtender sein als die von BubbleSort.

4 QuickSort



Lernziele

Nach der Bearbeitung dieses Kapitels

- verstehen Sie die Funktionsweise eines der schnellsten Sortierverfahren.
- können Sie die wichtigsten Begriffe erklären, welche in Zusammenhang mit diesem Verfahren von Bedeutung sind.

4.1 Die Idee



Der Sommer war lang und heiss. Willi konnte seine Mistkugelsammlung um viele schöne Exemplare erweitern. Leider hat er in seinem Sammeleifer die Ordnung ein bisschen vernachlässigt. Wieder einmal liegen die Kugeln unsortiert in einer Reihe. Nun möchte Willi die Ordnung wiederherstellen. Aber so viele Kugeln sortieren? Willi seufzt bei dem Gedanken. Da fällt sein Blick auf eine besonders schöne Kugel. Sie ist etwa von mittlerer Grösse und etwas dunkler als die anderen. An welcher Position

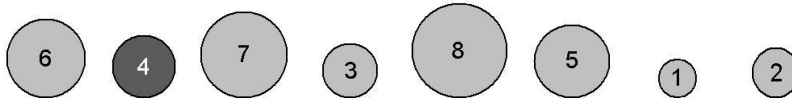
wird wohl diese Kugel stehen, wenn einst alle sortiert sind? Wahrscheinlich ungefähr in der Mitte. Und wenn ich es genau wissen möchte? sinniert Willi. Er überlegt eine Weile. Plötzlich steht er auf, nimmt seine Lieblingskugel und platziert sie ungefähr in der Mitte der

Kugelreihe. Dann sucht er alle Kugeln heraus, welche kleiner sind als die Lieblingskugel und schafft sie auf die linke Seite. Anschliessend bringt er alle Kugeln, die grösser sind als seine Lieblingskugel, auf die rechte Seite. Moment mal, überlegt sich Willi. Nun liegen also alle kleineren Kugeln links von meiner Lieblingskugel und alle grösseren rechts von ihr. Das bedeutet doch nichts Anderes als dass meine Lieblingskugel jetzt schon an ihrem endgültigen Platz liegt! Ob ich wohl damit schon einen kleinen Beitrag zur Sortierung geleistet habe?

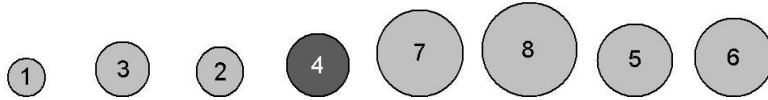
fragt sich Willi und betrachtet lange seine Mistkugelreihe.

Da kommt Willis Freund, der Frosch, des Weges. "Na, wieder mal am Sortieren?", fragt der Frosch scherzhaft. "Leider komme ich nicht weiter", klagt Willi. "Ich habe diesen Sommer so viele Kugeln gesammelt. Ich glaube ich schaffe es nicht, sie je in die richtige Reihenfolge zu bringen." "Aber so schlecht sieht das doch gar nicht aus", sagt der Frosch. "Du hast ja schon einen Anfang gemacht. Oh, und was ist denn das dort für ein Prachtexemplar?" fragt der Frosch und zeigt auf Willis Lieblingskugel. "Ja, das ist eine besonders schöne. Sie ist mir vorhin aufgefallen, und da habe ich - halb aus Spass, halb aus Verzweiflung - die anderen Kugeln so angeordnet. Aber sortiert sind sie deswegen noch lange nicht.", seufzt Willi. "Weisst du was?", sagt der Frosch nach einer kurzen Pause. "Ich helfe dir. Geteilte Arbeit ist halbe Arbeit. Ich sortiere den linken Teil deiner Reihe, also den mit den kleinen Kugeln, und du machst das gleiche mit dem rechten Teil, dort wo die grossen liegen. Und deine Lieblingskugel lassen wir schön brav an ihrem Platz liegen, damit sie ja nicht kaputtgeht." "Danke, das ist sehr nett von dir", sagt Willi. "Aber wie willst du das anstellen?" "Weiss ich noch nicht so genau", sagt der Frosch, "aber vielleicht hole ich noch meine Freundin, die Schnecke, zu Hilfe...".

Die Ausgangslage (unsortiert):



Nach Willis Umstellung: Alle Kugeln kleiner als 4 sind links von der 4, alle grösseren rechts.



Die Überlegungen von Willi und dem Frosch führen uns zu **QuickSort**. Wir werden bald herausfinden, ob dieses Verfahren seinen Namen verdient hat (*quick (engl.) = schnell*).

4.2 Der Algorithmus



Algorithmus

Schritt	Was tun
(1)	Wähle ein beliebiges, idealerweise ungefähr mittelgrosses Element aus dem Array.
(2)	Ordne den Array so um, so dass sich alle Elemente, die kleiner sind als das ausgewählte, links von diesem befinden und alle, die grösser sind, rechts desselben.
(3)	Sortiere den Teil-Array links des gewählten Elementes.
(4)	Sortiere den Teil-Array rechts des gewählten Elementes.

Natürlich gibt es dazu noch einige Fragen zu beantworten bzw. Teilprobleme zu lösen:

Frage 1: Was bedeutet "ungefähr mittelgross"? Wie finden wir ein solches Element?

Das exakt mittlere Element eines Arrays heisst **Median**. Das ist dasjenige Element, zu dem es gleich viele kleinere wie grössere Elemente gibt. Er ist nicht zu verwechseln mit dem arithmetischen Mittel aller Elemente! Den Median eines unsortierten Arrays zu bestimmen ist in der Tat relativ aufwändig.



4.2.1 Aufgabe

Finden Sie den Median der folgenden Reihen:

- a) 3 5 6 2 7 2 4
- b) 3 6 4 11 24 3 6 7 1 5

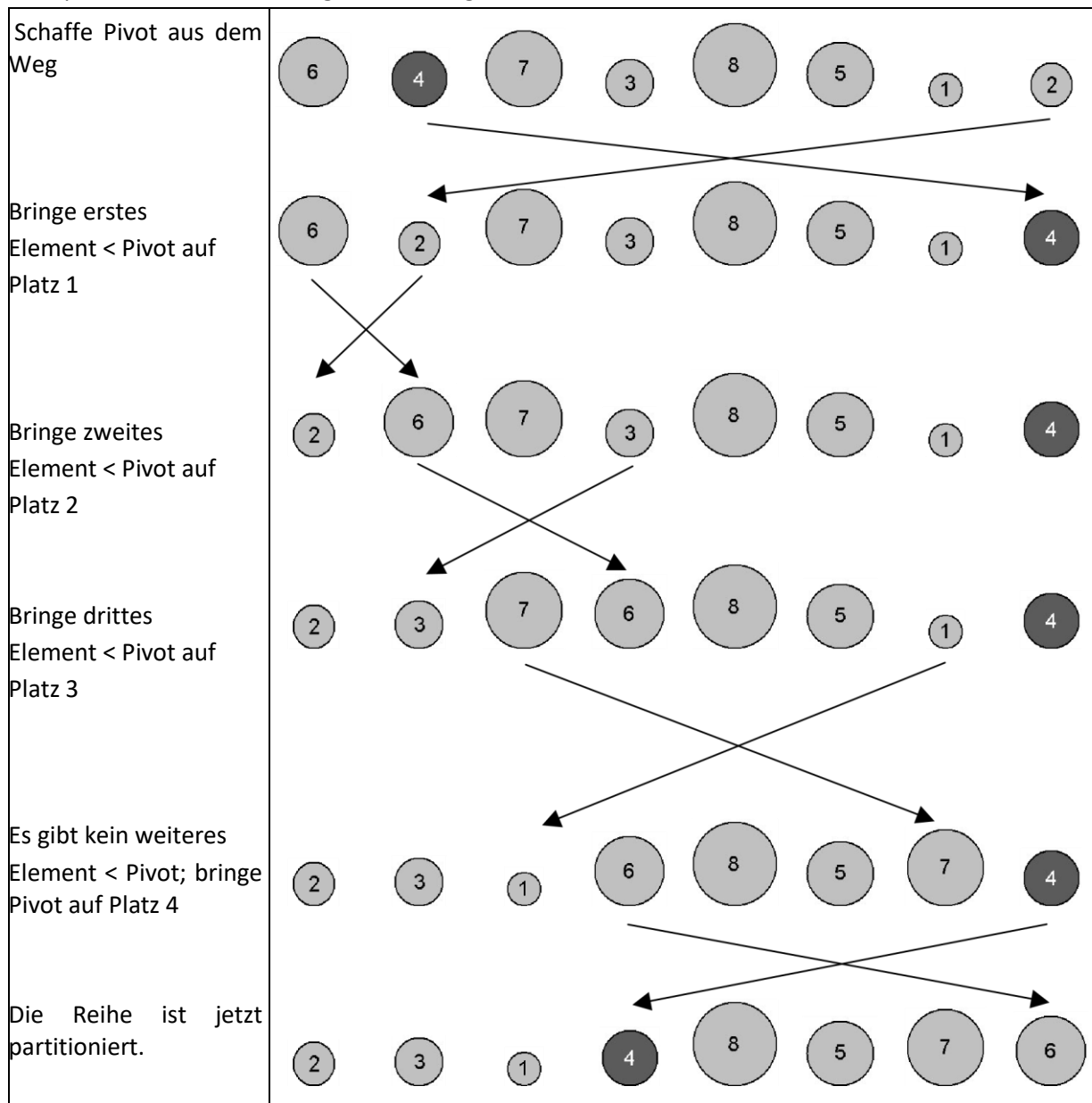
Für die Programmierung von QuickSort brauchen wir uns zum Glück um die Bestimmung des Medians keine Sorgen zu machen. Wir können ein *beliebiges* Element wählen. Der Algorithmus funktioniert trotzdem! Und jetzt noch ein Fachbegriff: Das ausgewählte "Lieblingselement" nennt man **Pivot-Element** oder kurz **Pivot**.

Frage 2: Wie können wir den Array so umordnen, dass sich links des ausgewählten Elementes alle kleineren und rechts davon alle grösseren Elemente befinden?

Auch für diesen Vorgang gibt es einen Fachbegriff: Man nennt ihn **Partitionierung**. Die Partitionierung lässt sich zum Beispiel wie folgt erledigen: Zuerst schaffen wir das Pivot Element aus dem Weg, indem wir es ganz auf die rechte Seite bringen.

Dann suchen wir im ganzen Array (ausser natürlich im letzten Feld) alle Elemente, die kleiner sind als das Pivot-Element und füllen sie von links her auf, indem wir sie mit dem entsprechenden Element vertauschen. Zuletzt bringen wir das Pivot-Element an die richtige Position.

Wir spielen die Partitionierung an Willis Kugelreihe durch:

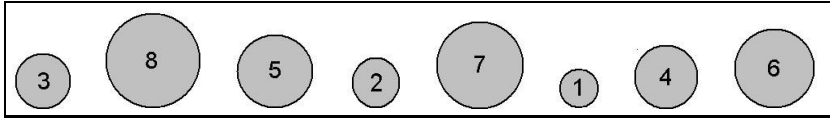


Wir haben hier ein anderes Ergebnis bekommen als Willi. Er hat ja die Partitionierung auch nicht genau nach unserem Verfahren durchgeführt. Natürlich sind beide Ergebnisse richtig.



4.2.2 Aufgabe

Bestimmen Sie für die folgende Reihe zwei verschiedene Pivot-Elemente ihrer Wahl und führen Sie für jedes die Partitionierung gemäss obigem Verfahren durch:



Nun können wir den Algorithmus für die Partitionierung formulieren. Wir bezeichnen die Schritte mit (2.1), (2.2) usw., da es sich hier um eine Verfeinerung von Schritt (2) des obigen Algorithmus handelt:



Algorithmus

Schritt	Was tun
(2.1)	Bringe das Pivot-Element durch Vertauschen an die letzte Array-Position.
(2.2)	Setze eine Variable k für die Einfügeposition auf die Position ganz links.
(2.3)	Wähle das erste Array-Element.
(2.4)	Ist das gewählte Element kleiner als das Pivot-Element? Wenn ja vertausche es mit dem Element an der Position k und erhöhe dann k um 1.
(2.5)	Gibt es rechts neben dem ausgewählten Element (ausser dem Pivot) noch weitere Elemente? Wenn ja, wähle das nächste aus und gehe zu (2.4).
(2.6)	Bringe das Pivot-Element durch Vertauschen an die Position k.

Hier nochmals die Partitionierung von Willis Beispiel, dieses Mal allerdings mit Zahlen und streng nach Anleitung:

Schritt	Was tun	k	Ergebnis dieses Schrittes							
	Ausgangslage		6	4	7	3	8	5	1	2
(2.1)	Bringe Pivot an letzte Stelle		6	2	7	3	8	5	1	4
(2.2)	Setze Einfügeposition k auf 1	1	6	2	7	3	8	5	1	4
(2.3)	Wähle erstes Element	1	6	2	7	3	8	5	1	4
(2.4)	Gewähltes Element ist nicht kleiner als Pivot, also nichts tun	1	6	2	7	3	8	5	1	4
(2.5)	Wähle nächstes Element	1	6	2	7	3	8	5	1	4
(2.4)	kleiner als Pivot, also vertauschen mit Pos. 1 und dann k um 1 erhöhen	2	2	6	7	3	8	5	1	4
(2.5)	Wähle nächstes Element	2	2	6	7	3	8	5	1	4
(2.4)	Gewähltes Element ist nicht kleiner als Pivot, also nichts tun	2	2	6	7	3	8	5	1	4
(2.5)	Wähle nächstes Element	2	2	6	7	3	8	5	1	4
(2.4)	kleiner als Pivot, also vertauschen mit Pos. 2 und dann k um 1 erhöhen	3	2	3	7	6	8	5	1	4
(2.5)	Wähle nächstes Element	3	2	3	7	6	8	5	1	4
(2.4)	Gewähltes Element ist nicht kleiner als Pivot, also nichts tun	3	2	3	7	6	8	5	1	4
(2.5)	Wähle nächstes Element	3	2	3	7	6	8	5	1	4
(2.4)	Gewähltes Element ist nicht kleiner als Pivot, also nichts tun	3	2	3	7	6	8	5	1	4
(2.5)	Wähle nächstes Element	3	2	3	7	6	8	5	1	4
(2.4)	kleiner als Pivot, also vertauschen mit Pos. 3 und dann k um 1 erhöhen	4	2	3	1	6	8	5	7	4
(2.5)	Es gibt kein weiteres Element mehr	4	2	3	1	6	8	5	7	4
(2.6)	Bringe Pivot an Position 4	4	2	3	1	4	8	5	7	6
	Fertig		2	3	1	4	8	5	7	6



4.2.3 Aufgabe (4.3)

Spielen Sie den Partitionierungs-Algorithmus anhand der folgenden Arrays und der gegebenen Pivot-Elemente durch. Füllen Sie dazu wie oben eine Tabelle aus:

- a) 5 7 11 3 8 6 9 2, Pivot = 7
 b) 5 8 4 6 3 8 3 1, Pivot = 3



4.2.4 Aufgabe (4.4)

Entwickeln Sie innerhalb ihres Sortierprogramms eine Funktion zur Partitionierung eines Arrays bei gegebenem Pivot-Element:

```
int partitionierenGanz(int[] a, int pivotIndex)
```

Diese Funktion bekommt als Parameter den Array und den Index des Pivots. Sie soll den Index des Pivots nach der Partitionierung, also seine endgültige Position, zurückgeben.

Wir werden diese Funktion später benötigen, um den ganzen QuickSort-Algorithmus zu bauen. Verwenden Sie die weiter oben entwickelten Funktionen zum Vergleichen und Vertauschen zweier Elemente. Auf diese Weise bereiten Sie ihren Code bereits für die Aufwandmessung vor. Testen Sie die Funktion an einigen Beispielen.

Eigentlich benötigen wir eine Funktion, welche nicht den ganzen Array partitioniert, sondern nur einen bestimmten Ausschnitt daraus. Erweitern Sie die obige Funktion um zwei Parameter, welche den zu partitionierenden Ausschnitt festlegen:

```
int partitionieren(int[a], int pivotIndex,  
                  int vonIndex, int bisIndex)
```

Sie können davon ausgehen, dass $\text{vonIndex} \leq \text{pivotIndex} \leq \text{bisIndex}$. Testen Sie ihre Funktion an einigen Beispielen.

Nun noch zur wohl alles Entscheidenden

Frage 3: Wie sortieren wir den linken und den rechten Teil-Array?

Die Antwort mag zugleich banal und verblüffend erscheinen: Natürlich auch mit QuickSort! Konkret bedeutet das, dass wir zur Sortierung des linken und des rechten Teil-Arrays wieder QuickSort aufrufen. Jeder Teil-Array wird dann wieder in zwei Teile geteilt, welche wieder in zwei Teile geteilt wird und so weiter. Diese Strategie, ein grosses Problem kleinere Einheiten zu zerlegen und diese nacheinander abzuarbeiten, wird "*Divide et impera!*" genannt. Das ist lateinisch, auf Deutsch heisst es "*Teile und herrsche!*".

QuickSort in dieser Form ist also ein **rekursives Verfahren**. Das Abbruchkriterium der Rekursion ist dann erfüllt, wenn ein Array der Grösse 1 sortiert werden soll. Da gibt es nämlich nichts zu tun.



4.2.5 Aufgabe

Mit der Partitionierung haben Sie einen wesentlichen Teil von QuickSort selbst entwickelt. Den Rest bekommen Sie geschenkt. Er besteht aus zwei Funktionen.

Die erste Funktion implementiert den eigentlichen rekursiven Algorithmus. Diese Funktion hat drei Parameter: den Array `a` und die Indexgrenzen des Ausschnitts, der sortiert werden soll:

```
void quickSortRek(int[] a, int vonIndex, int bisIndex)
{
    if (vonIndex >= bisIndex) { return; } // Abbruchbedingung
    int pivotIndex = (vonIndex + bisIndex) / 2; // Pivot-Wahl
    int pivotIndexDef = partitionieren(a, pivotIndex,
                                      vonIndex, bisIndex);
    quickSortRek(a, vonIndex, pivotIndexDef - 1);
    quickSortRek(a, pivotIndexDef + 1, bisIndex);
}
```

Die zweite Funktion kann von aussen ganz normal aufgerufen werden. Sie enthält nur eine einzige Anweisung, nämlich den Aufruf von `quickSortRek()` mit dem ganzen Array `a`:

```
void quickSort(int[] a) {
    quickSortRek(a, 0, a.Length - 1);
}
```

Bauen Sie den QuickSort-Algorithmus in ihr Sortierprogramm ein, indem Sie über einen neuen Button "QuickSort" die Funktion `quickSort()` aufrufen.



4.2.6 Aufgabe

Implementieren Sie den QuickSort-Algorithmus. Wie schon beim rekursiven SelectionSort sollen zwei Funktionen erstellt werden:

Erstens die rekursive Funktion

```
void quickSortRek(int[] a, int vonIndex, int bisIndex)
```

Sie implementiert den eigentlichen rekursiven QuickSort-Algorithmus. Diese Funktion hat drei Parameter: Den Array `a` und die zwei Indizes `vonIndex` und `bisIndex`, welche festlegen, welcher Bereich des Arrays sortiert werden soll. Wir erinnern uns, dass diese Funktion ja auch Teil-Arrays sortieren können muss. Diese Funktion soll die vorhin entwickelte Funktion zur Partitionierung aufrufen.

Zweitens die Funktion

```
void quickSort(int[] a)
```

Sie kann von aussen ganz normal aufgerufen werden. Sie enthält nur eine einzige Anweisung, nämlich den Aufruf von quickSortRek(), wobei der Array a sowie der tiefste und der höchste Index von a übergeben werden.

Bauen Sie den QuickSort-Algorithmus in ihr Sortierprogramm ein, indem Sie ihn über einen neuen Button "QuickSort" aufrufen.

4.3 Aufwandmessungen



4.3.1 Aufgabe

Sortieren Sie Zufalls-Arrays verschiedener Grössen mit QuickSort und tragen Sie die Anzahl Vergleiche und Vertauschungen in eine Tabelle ein.

Anzahl Vergleiche					
Sortierverfahren	Arraygrösse				
	10	30	100	300	1000
QuickSort					

Anzahl Vertauschungen					
Sortierverfahren	Arraygrösse				
	10	30	100	300	1000
QuickSort					



4.3.2 Zusatzaufgabe

Gegeben ist folgende Variante von QuickSort:

```
void quickSortRek2(int[] a, int vonIndex, int bisIndex)
{
    if (vonIndex >= bisIndex) { return; }
    int ix = (vonIndex + bisIndex) / 2;
    int x = a[ix];      int l = vonIndex;
    int r = bisIndex;
    do {
        while (istGroesser(x, a[l])) { l++; }
        while (istGroesser(a[r], x)) { r--; }
        if (l <= r)
        {
            vertauschen(a, l, r);
            l++;
            r--;
        }
    } while (l <= r);
    quickSortRek2(a, vonIndex, r);
    quickSortRek2(a, l, bisIndex);
}
```

Bauen Sie auch diese Variante in ihr Sortierprogramm ein (Button "QuickSort2") und vergleichen Sie die beiden Varianten bezüglich Anzahl Vergleiche und Vertauschungen. Können Sie die Unterschiede erklären?



4.3.3 Zusatzaufgabe

Programmieren Sie eine Funktion zur Ermittlung des Medians eines Arrays.

4.4 Zusammenfassung

Hier nochmals eine Zusammenfassung der Funktionsweise von Quicksort: Wir wählen ein Element des Arrays und nennen dieses *Pivot-Element*. Anschliessend führen wir eine sogenannte *Partitionierung* durch, indem wir den Array so umordnen, dass links vom Pivot nur kleinere Elemente, rechts von ihm nur grössere stehen. Im Idealfall, nämlich wenn wir als Pivot gerade den *Median* auswählen würden, würde auf diese Weise der Array gerade in zwei gleich grosse Hälften aufgeteilt. Die Bestimmung des Medians ist allerdings aufwändig. Glücklicherweise funktioniert das Verfahren auch dann, wenn wir ein *beliebiges* Element als Pivot wählen. Man beachte, dass die beiden Teil-Arrays links und rechts des Pivots nach dem Partitionierungsvorgang immer noch unsortiert sind. Die Sortierung des linken und des rechten Teil-Arrays erfolgt nach dem Prinzip "*Teile und herrsche!*": Wir wenden das gleiche Verfahren auf die beiden Teil-Arrays an, von denen natürlich jeder kleiner ist als der ganze Array. Indem wir diesen Mechanismus *rekursiv* fortsetzen, bis wir nur noch Arrays der Grösse 1 zu sortieren haben, erreichen wir schliesslich unser Ziel, nämlich die Sortierung des ganzen Arrays.

Wir ahnen bereits, dass QuickSort bezüglich Laufzeit besser abschneidet als die beiden vorher behandelten Verfahren. Im nächsten Kapitel werden wir die drei Verfahren miteinander vergleichen und die Ergebnisse anschaulich darstellen.

5 Vergleich der Verfahren



Lernziel

Nach der Bearbeitung dieses Kapitels

- wissen Sie, welches der hier besprochenen Sortierverfahren das schnellste ist.

5.1 Daten sammeln und darstellen



5.1.1 Aufgabe

Sie haben zu jedem Sortierverfahren eine Tabelle mit der Anzahl Vergleiche und Vertauschungen für verschieden grosse Arrays erstellt. Nun wollen wir die drei Verfahren miteinander vergleichen.

- a) Führen Sie die Tabellen der Anzahl Vergleiche und Vertauschungen zusammen und erweitern Sie sie - soweit die Rechnerleistung es zulässt - um Spalten für die Arraygrößen 3000, 10000, 30000 und 100000:

Anzahl Vergleiche										
Sortierverfahren	Arraygrösse									
	10	3	10	30	100	300	10000	30000	100000	
	0	0	0	0	0	0				
Anzahl Vertauschungen										
Sortierverfahren	Arraygrösse									
	10	3	10	30	100	300	10000	30000	100000	
	0	0	0	0	0	0				
BubbleSort										
BubbleSort Opt										
SelectionSort										
QuickSort										
BubbleSort										
BubbleSort Opt										
SelectionSort										
QuickSort										

- b) Erstellen Sie mit Hilfe eines Tabellenkalkulations-Programmes (z.B. Excel) Diagramme aus den beiden obigen Tabellen. Beachten Sie, dass die Rubriken-Achse durch die vorgegebene Wahl der

Arraygrößen (10, 30, 100, 300,...) eine logarithmische Skala besitzt. Wählen Sie auch für die y-Achse eine logarithmische Skala. Kommentieren Sie das Ergebnis.

c) Unter der Annahme, dass eine Vertauschung ungefähr gleich viel Rechenzeit braucht wie ein Vergleich, macht die Summe dieser beiden Größen eine Aussage über die Effizienz eines Verfahrens als Ganzes. Erstellen Sie ein Diagramm mit der Summe von Anzahl Vergleichen und Vertauschungen.

5.2 Zusammenfassung

Von den vielen Sortierverfahren, die es gibt, haben wir drei ausführlich besprochen, implementiert und miteinander verglichen. Diese Verfahren funktionieren alle korrekt, unterscheiden sich aber in ihrem Laufzeit-Aufwand um mehrere Größenordnungen. Während bei kleinen Arrays die Unterschiede kaum spürbar sein dürften, zeigen sich die Differenzen bei grossen Arrays umso mehr. Für den praktischen Einsatz ist es äusserst wichtig, einen schnellen Algorithmus zu wählen. Von **BubbleSort** ist also dringend abzuraten. **SelectionSort** ist zwar wesentlich schneller als **BubbleSort**, für grosse Arrays aber trotzdem nicht zu empfehlen. Mit **QuickSort** haben wir eines der schnellsten Sortierverfahren kennen gelernt, die es überhaupt gibt.